вия будет проработана и освоена. Доля значимости перейдет уже в показатели трудозатратности и достоверности сведений.

- Возможность проведения руководством ОГВ мониторинга процесса оказания услуги (Мг). К третьему этапу значимость, а соответственно и доля значимости показателя в общей оценке эффективности, будет небольшой за счет того, что основная часть статистических отчетов о работе по заявкам и запросам будет формироваться автоматизированными информационными системами с большей по сравнению с сотрудником точностью. Доля значимости перейдет уже в показатели трудозатратности и достоверности сведений.
- Достоверность сведений (Дс). При переводе проекта на последующие этапы доля значимости показателя сначала резко снижается относительно остальных оцениваемых показателей, а на третьем этапе резко возрастает. Это происходит вследствие того, что на втором этапе сотрудники начинают получать более достоверные сведения из ОГВ и ОМСУ по юридически значимым каналам. На третьем этапе доля значимости резко возрастает, но не достигает первичного состояния, так как сотрудника больше заботят объем и достоверность содержания запросов, чем остальные оптимизационные критерии.

На основании изложенного можно отметить следующее.

Для исследования была выбрана типовая модель жизненного цикла проекта по стандарту ISO-15288. Применение этой модели обусловлено практическим переходом к проектному управлению по международным стандартам, что придает дополнительное разнообразие возможностям применения дополнительных технологий анализа и менеджмента при проектном управлении [6].

Математическим методом доказана эффективность реализации проекта. На основе реальных и синтезированных точек зрения был составлен портрет эффективности проекта в рамках этапов его реализации. Кроме того, на основании полученных оценок эффективности спрогнозированы результаты проекта для тактического и стратегического менеджмента проекта.

Литература

1. Аленцев В., Самарин А. Эталонная модель BPM // Открытые системы. 2009. № 1. URL: http://www.osp.ru (дата обращения: 26.08.2013).

2. Швецов А.Н. Агентно-ориентированные системы: от формальных моделей к промышленным приложениям: Всерос. конкурсный отбор обзорно-аналит. статей по приоритет. направлению «Информационно-телекоммуникационные системы». 2008. 101 с.

3. OMG. BPMN (Business Process Modeling Notation) v.1.0, 2006. URL: http://www.omg.org/technology/documents/bms_spec_catalog.htm (дата обращения: 26.08.2013).

4. Ципес Г.Л., Товб А.С. Управление проектами: стандарты, методы, опыт. М.: Олимп-Бизнес, 2003. 240 с.

5. Fingar P., Smith H. Business Project Management. The third wave. 1st ed., Meghan-Kiffer Press, Tampa, 2003, 312 p.

6. Sowa J.F., Zachman J.A. Extending and Formalizing the Framework for Information System Architecture. IBM System Journ., 1992, vol. 31, no. 3.

References

1. Alentsev V., Samarin A. *Otkrytye sistemy* [Open Systems], 2009, no. 1. Available at: http://www.osp.ru (accessed 26 August 2013)

2. Shvetsov A.N. *Vserossiyskiy konkursny otbor obzorno-analiticheskikh statey po prioritetnomu napravleniyu "Informatsionno-telekommunikatsionnye sistemy"* [All-Russian competitive selection of overview and analytical articles on the priority area "Information and Telecommunication Systems"], 2008.

3. *OMG. BPMN (Business Process Modeling Notation) v.1.0*, available at: http://www.omg.org/technology/documents/bms_spec_catalog.htm (accessed 26 August 2013).

4. Tsipes G.L., Tovb A.S. *Upravlenie proektami. Standarty, metody, opyt* [Project Management: standards, methods, experience]. Moscow, Olimp-Business Publ., 2003 (in Russ.).

5. Fingar P., Smith H. *Business Project Management. The third wave*. 1st ed., Meghan-Kiffer Press, Tampa, 2003.

6. Sowa J.F., Zachman J.A. *IBM System Journ.*, 1992, vol. 31, no. 3.

УДК 004.451.7.031.43

ПОСТРОЕНИЕ OLAP-КУБОВ С ПОМОЩЬЮ СТАНДАРТНЫХ СРЕДСТВ РАЗРАБОТКИ WEB-ПРИЛОЖЕНИЙ

(Работа выполнена в рамках ФЦП по проекту «Программный комплекс и облачный сервис для автоматизации обобщения и анализа финансово-экономической информации о деятельности предприятий и обеспечения принятия управленческий решений», ГК № 14.527.12.0020)

В.Н. Чибисов, директор по информационным технологиям;
И.А. Суслов, старший специалист
(МХК «ЕвроХим», ул. Дубининская, 53, стр. 6, г. Москва, 115054, Россия, vladimir.chibisov@eurochem.ru, ivan.suslov@eurochem.ru);
И.А. Маркилов, начальник отдела
(«МБК Груп», ул. Петровка, 17, стр. 1, г. Москва, 107031, Россия, igoram@mbcgroup.ru)

Описывается построение системы, работающей с многомерными аналитическими пространствами для обработки информации (OLAP-кубов данных), с помощью свободно распространяемых бесплатных инструментальных и программных средств для разработки web-приложений. Рассматриваются вопросы построения структуры БД, разработки системы кодирования и основанного на ней языка обращения к многомерному пространству, а также построения и проектирования транслятора, преобразующего выражения, написанные на языке обращений к многомерному пространству, в язык запросов к БД (язык запросов SQL). Использование описанного в данной статье подхода позволяет осуществлять разработку и построение OLAP-кубов данных на стандартных средствах разработки web-приложений без привлечения дорогостоящих специализированных программных средств для работы с многомерными аналитическими пространствами (OLAP-кубами данных).

Ключевые слова: *OLAP, автоматизация процесса, web-приложения, бизнес-анализ, трансляция.*

BUILDING OLAP-CUBES USING STANDARD WEB-DEVELOPMENT APPLICATIONS

Chibisov V.N.*, IT-director;* ***Suslov I.A.****, senior engineer*
(EuroChem, Dubininskaya St., 53/6, Moscow, 115054, Russian Federation,
vladimir.chibisov@eurochem.ru, ivan.suslov@eurochem.ru);
Markilov I.A.*, head of department (Middle Business Consulting Group,*
Petrovka St., 17/1, Moscow, 107031, Russian Federation, igoram@mbcgroup.ru)

Abstract. This paper addresses building of a system working with multi-dimensional analytic spaces used to process analytical data (OLAP-data cubes) by means of shareware, free tools and software to develop web-applications. The paper considers the issues related to development of a database structure, the encoding system and the language used to address to the multi-dimensional space, as well as building and design of a translator converting the expressions written in the language used to address to the multi-dimensional space into the language used to make database queries (SQL data-query language). Application of the approach described in this paper allows designing and building OLAP-data cubes using conventional means for web-application development without using expensive, special-purpose software to work with multi-dimensional analytic spaces (OLAP-data cubes).

Keywords: OLAP, process automation, web-applications, business analysis, broadcast.

Современные технологии (в частности, web-технологии) оказывают все большее влияние на бизнес и внедряются повсеместно. Так как большая часть населения цивилизованных стран активно использует Интернет как средство общения и ведения бизнеса, это влияет на наличие в сети потенциальных клиентов. Использование всех имеющихся инструментов для общения и коммуникации, для эффективного развития собственного бизнеса и продвижения своего бренда или просто для творческой реализации в сети – жизненная необходимость, возможность экономить деньги и время на выполнение определенных задач, которые раньше требовали больше человеческих и аппаратно-программных ресурсов.

Web-технологии позволяют пользователям взаимодействовать с системой через стандартные средства работы с Интернетом и сетями-браузерами, что выгоднее использования дополнительных клиентских приложений, устанавливаемых на рабочее место пользователя. За счет этого такие системы очень мобильны, так как работать с ними можно через любые удаленные и стационарные устройства, имеющие выход в Интернет.

Система, построенная с использованием web-технологий, не имеет тяжелых клиентских и серверных приложений. Конечно, такие системы работают немного медленнее своих аналогов, но от систем бизнес-анализа не требуется работа в режиме реального времени. Бизнес-анализ выполняется по временным отрезкам: минимально необходимым отрезком являются сутки. Мобильность, достигаемая за счет возможности работы с любого устройства и без предварительных необходимых настроек, позволяет аналитикам производить анализ и принимать решения на его основе из любой точки планеты при наличии Интернета и устройства для выхода в него.

Большинство современных BI-систем работают с использованием специализированных программных средств для работы с OLAP-кубами данных. Но, поскольку построение OLAP-кубов возможно на обычных реляционных БД [1], в данной статье предлагается подход для построения OLAP-кубов на основе свободно распространяемых инструментальных и программных web-средств (Python, Django, СУБД MySQL и др.). Естественно, такие средства имеют меньше возможностей, чем их платные аналоги. Например, используемая СУБД MySQL не предусмотрена для работы с многомерными пространствами (с OLAP-кубами). Основная сложность построения OLAP-кубов заключается в отсутствии простого, интуитивно понятного средства для обращения к многомерному пространству данных (языка запросов по аналогии с MDX) [2, 3].

Оригинальный авторский подход для построения многомерного пространства предполагает последовательную разработку организованной по принципам OLAP структуры БД MySQL, системы кодирования для всех информационных объектов, языка обращения к многомерному пространству, транслятора для организации выполнения запросов к многомерному пространству.

Схематическое изображение подхода дано на рисунке 1.

Первоначальными шагами построения OLAP-кубов являются создание объектной модели будущей системы и формирование на ее основе структуры БД, сформированной по принципам

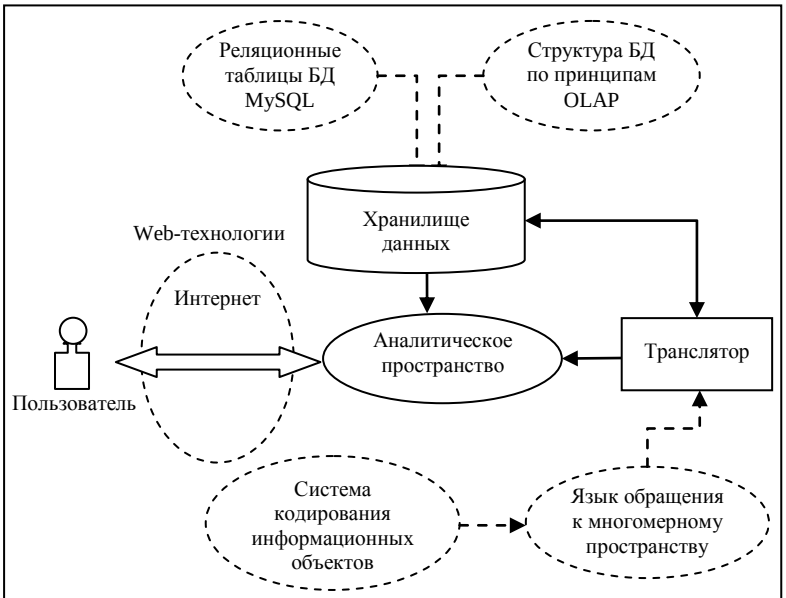


Рис. 1. Схематическое изображение подхода к построению системы

OLAP. Построенная таким образом БД будет хранилищем данных, содержащим информацию по многомерным аналитическим пространствам.

Следующие шаги построения OLAP-кубов – выделение объектов будущей системы, которые будут участвовать в процессе обработки данных, и определение для них системы кодирования.

На основании построенной системы кодирования информационных объектов разрабатывается такой язык обращения к многомерному пространству OLAP-кубов, который будет максимально наглядным и простым для того, чтобы обычный пользователь мог самостоятельно формировать запросы к многомерному пространству данных и понимал, что он должен получить в качестве результата запроса.

Для преобразования элементов выражения-запроса в язык, понятный СУБД, хранящей данные, создается программный транслятор.

Организация структуры БД по принципам OLAP

Для построения структуры БД по принципам OLAP необходимо создать объектную модель системы (то есть выделить основные объекты, с которыми будет вестись работа, и определить их взаимосвязи), а затем на ее основе определить количество измерений в строящемся многомерном пространстве.

Предположим, что была разработана объектная модель будущей системы и выделены основные оси многомерного куба – измерения пятимерного пространства, такие как периоды, типы данных, компании, показатели, сегменты.

На основании определенных измерений строится БД в соответствии с принципами OLAP [1–4]. При создании web-приложений, сайтов и блогов используется MySQL как наиболее распространенная СУБД. Поскольку она является реляционной БД, используется реализация ROLAP (рис. 2).

В ROLAP основными составляющими архитектуры БД являются таблицы фактов и измерений [1]. В таблице фактов обычно содержатся сведения об объектах или событиях, совокупность которых будет подвергнута анализу. В таблице измерений размещены постоянные данные, по которым выполняются анализ и построение аналитических пространств. Таблицы фактов содержат числовые значения ячеек куба данных, а остальные определяют их многомерную совокупность измерений. Представление многомерной информации с помощью звездообразных реляционных моделей устраняет проблему оптимизации хранения разреженных матриц, остро стоящую перед многомерными СУБД, в которых проблема разреженности решается специальным выбором схемы хранения.

Для обеспечения постоянной производительности системы при различных объемах обрабатываемой информации структура БД организуется таким образом, что количество внесенной информации и созданных внутренних объектов не накладывает ограничений на производительность. Это достигается за счет создания множества таблиц фактов и хранения обработанных частично агрегированных данных [3–5].

Построенная таким образом схема данных подразумевает несколько центральных таблиц фактов, связанных с таблицами измерений.

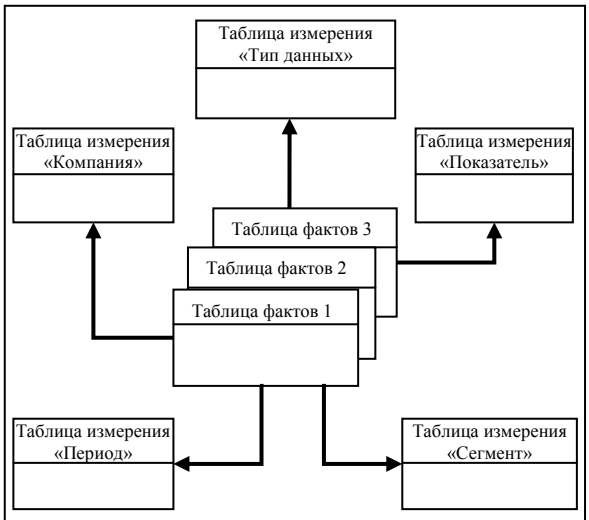


Рис. 2. Структура БД MySQL, построенной по принципам ROLAP

Разработка системы кодирования информационных объектов

Создание данной системы включает три последовательных этапа: идентификация объектов, их классификация и кодирование [6–8].

Под идентификацией подразумевается присвоение объекту уникального имени, признака или набора признаков, позволяющих однозначно выделить его из других объектов.

Классификация объектов используется для сбора, обработки и представления необходимой информации. Классификация информационных объектов подразумевает их систематизацию.

Кодирование представляет собой упорядоченное образование и присвоение кодов информационному объекту, позволяющее заменить несколькими знаками (символами) имена этих объектов. С помощью кодов обеспечивается возможность идентификации объектов в Комплексе SST максимально коротким способом, с помощью минимального числа знаков. Стремление к минимизации количества знаков, идентифицирующих объекты, способствует повышению эффективности сбора, учета, хранения, обработки информации.

Кодовое обозначение характеризуется алфавитом и структурой кода, числом знаков (длиной кода), методом кодирования.

На этапе классификации объектов необходимо соблюдать полноту охвата объектов рассматриваемой области, однозначность реквизитов, возможность включения новых объектов.

На этапе кодирования объектов разработанные коды должны удовлетворять следующим основным требованиям:

– однозначно идентифицировать информационные объекты;

– иметь минимальное число знаков, достаточное для кодирования всех объектов (признаков) заданного множества;

– иметь достаточный резерв для кодирования вновь возникающих объектов кодируемого множества;

– быть удобными для пользователей и для компьютерной обработки закодированных информационных объектов.

При выполнении первого этапа работ следует руководствоваться стандартным подходом к идентификации информационных объектов, который заключается в выделении ключевых наборов атрибутов для определенного вида объектов и сравнения их значений для определения совпадающих и похожих пар объектов. При этом необходимо учитывать, что одинаковые значения для одного набора ключевых атрибутов однозначно свидетельствуют о совпадении объектов, а наличие одинаковых значений для другого набора атрибутов позволяет сделать вывод только о похожести объектов.

В соответствии с примером по определению осей многомерного куба в процессе построения системы кодирования идентифицированы и классифицированы шесть видов объектов, с которыми работают пользователи: компании, типы данных, периоды, показатели, отчеты, сегменты. Для выделенных объектов можно построить систему кодирования [8], основанную на описанных выше правилах (табл. 1).

Таблица 1

Система кодирования информационных объектов Комплекса SST

Идентификация	Классификация	Кодирование	Пример кодировки
Компания	[код компании] = [международный код страны] [аббревиатура предприятия]	<2 буквы> <3 буквы>	RUNAK
Тип данных	[код типа данных]	<1 буква>	A
Период	[код даты] = [День] [Месяц] [Год]	<2 цифры> <2 цифры> <4 цифры>	01052013
Показатель	[Код показателя] = [Код формы] [Код строки]	<3 буквы> <3 цифры>	TBL010
Отчет	[код отчета] = [код даты] [код типа данных] [.] [код компании]	<8 цифр> <1 буква> <5 букв>	01012012A. RUNAK
Сегмент	[Код сегмента]	<3…20 букв>	ammophos

Разработка языка обращения к многомерному пространству

При обработке многомерных кубов данных большинство OLAP-продуктов использует MDX-запросы. Обычному пользователю трудно понимать и составлять запросы на языке MDX. Для этого необходимо знание таблиц БД в системе и языка SQL. В предлагаемом подходе на базе системы кодирования реализуется свой механизм построения запросов к многомерному пространству. Обработка этих запросов производится транслятором, который позволяет с помощью простых математических действий выполнить все необходимые преобразования данных. Для написания запросов к многомерному пространству пользователю необходимо знать лишь небольшое количество правил составления запросов: то, как он закодировал объекты системы, и простые математические и логические операции.

Будущая система настраивается таким образом, что пользователь при создании объектов сис-

темы (компании, периоды, отчеты и так далее) самостоятельно кодирует объекты в соответствии с разработанной системой кодирования. Язык запросов к многомерному пространству разрабатывается таким образом, что в качестве его инструкций и операторов используются только заданные коды информационных объектов, математические и логические преобразования (стандартные арифметические и логические операции). Следовательно, при построении отчетов ему не придется изучать сложный синтаксис MDX-запросов.

Приведем пример для сравнения: один и тот же запрос для получения месячного отчета (в июне 2012 года) по производству (за каждый день) конкретного вида продукции (нитроаммофоска) предприятием (с кодом RUNEV), составленный с помощью системы кодирования:

062012A.RUNEV.NIT110,

где 062012 – период; A – фактические данные; RUNEV – предприятие; NIT110 – код показателя,

и MDX-запрос:

SELECT {[Month]. [June, 2012]. [Day]} ON COLUMNS, {[Measures]. [Units]} ON ROWS FROM [Production]. [Actual] WHERE ([Product]. [NITRO], [Company]. [RUNEV]).

Очевидно, что для написания MDX-запросов необходимо иметь специальную подготовку, а для получения нужных данных в Комплексе SST пользователю достаточно помнить, как он назвал необходимые ему объекты.

Закодировав все объекты и соблюдая составленную иерархию кодирования, пользователь самостоятельно все описывает, создавая новые объекты. Пример иерархии кодирования: {период, тип данных, компания, показатель, сегмент}.

Пример запросов.

Использование различных сегментов:
01122013A.RUNEV.TBL110(S_ammophos)+011202013A.RUNEV.TBL110(S_karbamid).

Использование различных компаний:
01122013A.RUNEV.TBL110+011202013A.RUFOS.TBL110.

Использование различных паказателей:
01122013A.RUNEV.TBL110+RUNEV.BRS510(-1D).

Использование различных периодов:
01122013A.RUNEV.TBL110+01102013A.RUNEV.TBL110.

Использование различных типов данных:
01122013A.RUNEV.TBL110+01122013F.RUNEV.TBL110.

Комбинированное использование: различные периоды, типы данных, компании, показатели, сегменты:
01122013A.RUNEV.TBL110(S_ammophos)+011202013F.RUFOS.BRS510(S_karbamid).

Для обработки созданного языка определяются ***формальная грамматика*** и ***синтаксис***.

В соответствии с принятой системой кодирования и основными операциями над показателями составляется формальная грамматика [8], состоящая из терминального алфавита, нетерминального алфавита и правил грамматики. Пример терминального алфавита грамматики приведен в таблице 2.

Таблица 2

Терминальный алфавит формальной грамматики

	0	1	2	3	4	5	6	7	8	9	+
-	/	*	(	)	F.	M.	B.	O.	EPP	PP	-1
-1D	S_	a	b	c	d	e	f	g	h	i	j
k	l	m	n	o	p	q	r	s	t	u	v
w	x	y	z								

Пример нетерминального алфавита данной грамматики приведен в таблице 3.

Таблица 3

Нетерминальный алфавит формальной грамматики

Бизнес-выражение	Формула	Знак	Показатель
Тип данных	Код формы	Код строки	Модификатор
Строка	Число	Цифра	Буква
Код модификатора			

Приведем пример правил, определяющих синтаксис для данной грамматики. Пример описывает построение грамматики для формул бизнес-выражений (формулы математических операций над показателями) [1].

Правила формальной грамматики:

Бизнес-выражение = <Формула>
Формула = <Формула> <Арифметическая операция> <Формула>
Формула = (<Формула>)
Формула = <Показатель>
Арифметическая операция = + | - | / | *
Показатель = <Тип данных><Код формы><Код строки><Модификатор>
Тип данных = | F. | M. | B. | O.
Код формы = <Буква><Буква><Буква>
Код строки = <Цифра><Цифра><Цифра>
Модификатор = | (<Код модификатора>)
Код модификатора = B | E | Z | EPP | PP | -1 | -1D | S_<строка> | <Число> | -<Число>
Строка = <Буква>
Строка = <Строка><Буква>
Число = <Цифра>
Число = <Число><Цифра>
Цифра = 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9
Буква = a | b | c | d | e | f | g | h | i | j | k | l | m | n | o | p | q | r | s | t | u | v | w | x | y | z

Начальным терминалом данной грамматики является бизнес-выражение. Правила формальной грамматики полностью определяют синтаксис формул бизнес-выражений, и на их основе производится синтаксический анализ в трансляторе. Примеры формул в соответствии с формальной грамматикой и синтаксисом:

isn001
isn110 + abc120
(wtf155 * xyz992) / M.ksn345
isf402 + abc123 - (B.isf130(-1D) + isf120 * isf330(Z))

Построение транслятора

Работа транслятора обусловлена следующими тремя фазами обработки входной строки на исходном языке [9–12]:

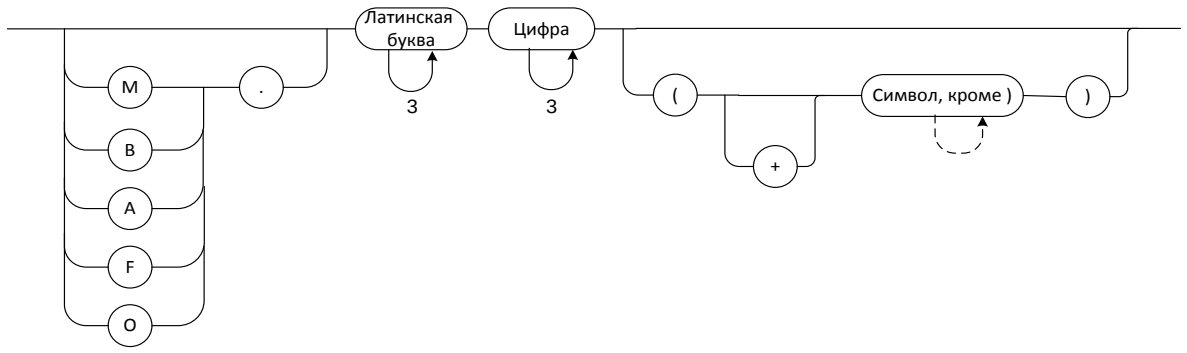


Рис. 3. Схема регулярного выражения для разбора бизнес-выражений

– фаза лексического анализа;

– фаза синтаксического анализа: распознавание синтаксической структуры и семантический разбор, в процессе которого осуществляются работа с таблицами, порождение промежуточного семантического представления или объектной модели языка;

– фаза генерации кода: семантический анализ компонентов промежуточного представления или объектной модели языка и перевод промежуточного представления или объектной модели в объектный код.

Лексический анализ – первая фаза процесса трансляции, предназначенная для группировки символов входной цепочки в более крупные конструкции, называемые лексемами. Обычно претендентами на роль лексем выступают элементарные конструкции языка. Полученные лексемы передаются синтаксическому анализатору.

Синтаксический анализатор – компонент транслятора, осуществляющий проверку исходных операторов на соответствие синтаксическим правилам и семантике данного языка программирования. Он осуществляет разбор исходного текста, используя поступающие лексемы, построение синтаксической структуры текста и семантический анализ с формированием объектной модели языка.

Для получения и выделения лексем и выполнения лексического и синтаксического анализов необходимо определить грамматику разбора входных строчек.

Для разработки транслятора нужно определить грамматику, на основании которой проводится лексический и синтаксический анализы показателей (грамматика языка для разбора определена на этапе построения языка). Для программирования работы транслятора данная грамматика переводится на язык регулярных выражений [10, 12]. Например, регулярное выражение для разбора бизнес-выражений, составленных в соответствии с приведенными выше правилами формальной грамматики и работами [13, 14], примет вид **((?:([MBAFO])\.)?([A-Z]{3})([0-9]{3})(?:\(\+?([^)]+)\))?)**.

Такое регулярное выражение, описывающее грамматику, можно представить схематично [13], изобразив конечный автомат, описывающий последовательность лексем (рис. 3).

Данный автомат является исходной грамматикой и используется для синтаксического анализа и выделения лексем для лексического анализа в программном коде.

Процесс генерации целесообразно выполнять как многоступенчатый [11, 12], поскольку многие алгоритмы оптимизации кода проще реализовать каждый отдельно и какой-то шаг оптимизации зависит от результата отработки другого шага.

Кроме того, при такой организации легко создать один компилятор, который будет создавать код для нескольких платформ, так как достаточно заменить последний шаг генерации кода.

Другими словами, формирование и генерацию результирующего целевого выражения лучше всего разбить на отдельные части (этапы). То есть формировать не сразу целиком все исходное выражение, а сначала небольшие куски кода, которые затем будут объединены в целевое выражение. При такой генерации целевых выражений можно организовывать конвейерное преобразование, когда различные части исходного выражения (на основании полученных во время лексического анализа лексем) генерируют части целевого выражения параллельно друг другу.

Суть процесса генерации заключается в том, чтобы преобразовать инструкции и операторы с разработанного языка на язык запросов к СУБД (язык запросов SQL).

Для эффективной генерации целевого выражения на вход транслятора должны подаваться основанные на формальной грамматике значения: коды временного среза (период), компании, типа данных, информационного среза (отчет, сегмент), показателя, а также формулы математических и логических преобразований (бизнес-выражения).

Но фактически для работы транслятора необходимо передавать только три параметра (табл. 4), так как некоторые параметры могут объединять в себе несколько значений. Это возможно благодаря правильной организации хранилища данных.

Таблица 4

Входные параметры в транслятор

Наименование	Кодовое обозначение	Тип данных	Описание
Список формул	expressions	list<string>	Список математических и логических преобразований (формул бизнес-выражений)
Список информационных срезов	reports	list<number>	Список объектов класса «Отчет»
Список сегментов	segments	list<number>	Список объектов класса «Сегмент»

Основой для генерации результирующего целевого выражения служат формулы, поскольку они задают источники данных, указывают и рассчитывают значения показателей. Разбор формул в лексическом и аналитическом анализаторах позволяет транслятору определить коды значений показателей и их координаты в таблицах фактов многомерного аналитического пространства ROLAP-куба [1, 2]. Информационные срезы и сегменты определяют координаты остальных осей в многомерном пространстве ROLAP-куба.

Для возможности использования данных из других временных периодов и других информационных срезов можно в язык обращения к многомерному аналитическому пространству добавить дополнительные элементы, модификаторы [1] – дополнительные атрибуты показателя, позволяющие использовать показатели в различных срезах данных или различных отчетных периодах.

При составлении формул бизнес-выражений, для расчетов показателей деятельности предприятий использование модификаторов можно реализовать, дописав его в скобках после показателя. Тогда синтаксис записи модификатора будет выглядеть следующим образом: **<Показатель> (<Модификатор>).**

Например, показатель **ISN010** с модификатором «**B**» запишется как **ISN010(B).**

С учетом правил возможной записи показателя (в соответствии с правилами грамматики), который состоит из **кода формы** и **кода строки** на данной форме, его синтаксис приобретет следующий вид: **<тип данных>.<код формы><код строки>(<модификатор>).**

Например, для системы бизнес-анализа можно использовать девять модификаторов:

B – вернет данные (показатель) из предыдущего временного среза за ближайший заполненный период;

E – вернет данные (показатель) последнего периода текущего временного среза;

Z – вернет данные (показатель) для всех сегментов данной формы, то есть вернет консолидирующий сегмент;

EPP – вернет данные (показатель) последнего периода предыдущего временного среза;

PP – вернет данные (показатель) периода предыдущего временного среза, аналогичного периоду текущего временного среза;

–1 – вернет данные (показатель) ближайшего заполненного периода (то есть периода, где есть данные); работает для всех временных срезов;

–1D – вернет данные (показатель) ближайшего заполненного периода (то есть периода, где есть данные) в текущем временном срезе;

S_<сегмент> – вернет данные (показатель) для определенного сегмента, то есть при указании модификатора определяется, из какого сегмента следует выбирать данные;

смещение – модификатор возвращает данные (показатель) в периоде, указанном смещением; при этом смещение может быть как положительным – вперед по шкале временных периодов, так и отрицательным – назад по шкале временных периодов, к более ранним периодам.

Обработка модификаторов и их значений происходит в процессе трансляции при формировании (генерации) целевого SQL-выражения. Модификаторы могут быть введены для формирования более эффективных SQL-запросов к БД с возможностью выборки данных из различных периодов и информационных срезов.

Транслятор необходимо запрограммировать (разработать) таким образом, чтобы в процессе трансляции формулы бизнес-выражений преобразовывались в SQL-выражение, пригодное для обработки в СУБД, проверялись показатели формул на корректность, определялись таблицы для выборки данных и выполнялись арифметические построения при создании SQL-запроса. Сформированные SQL-запросы отправляются на обработку в СУБД MySQL. Возвращаемым результатом таких запросов должны быть словари данных с рассчитанными и полученными массивами данных из многомерного аналитического пространства ROLAP-куба, которые в дальнейшем и будут обрабатываться системой.

Алгоритм реализации транслятора

Транслятор бизнес-выражений может быть реализован в виде алгоритма.

Шаг 1. Получение исходных данных для генерации SQL-выражения.

Для обработки и трансляции необходимо получить следующие данные:

– формулы бизнес-выражений (основной элемент при генерации);

– идентификаторы отчетностей для определения тех, в которых происходят выборки и вы-

числения (отчетность включает в себе сведения о типе данных, периоде и компании);

– идентификаторы сегментов для определения и выполнения вычислений для каждого отдельного сегмента.

Шаг 2. Подготовка части выходного SQL-выражения для расчета формул на уровне БД, определение арифметических операций с показателями (генерация, оптимизация части SQL-выражения и лексический анализ).

Выделение необходимых лексем из формул с помощью заданного регулярного выражения и построение части выходного SQL-выражения, отвечающего за проведение арифметических операций. Разбор выражения с помощью регулярного выражения (лексический анализ).

Шаг 3. Формирование части выходного SQL-выражения, определяющего список используемых в БД таблиц. Формирование списка используемых форм с помощью определенных на предыдущем шаге лексем, задействованных в формулах бизнес-выражений, и запись их в виде SQL-выражения. При этом происходят генерация части SQL-выражения и его оптимизация.

Шаг 4. Формирование части целевого выходного SQL-выражения, включающего в себя основную таблицу БД для выборки данных. При этом происходят генерация части SQL-выражения и его оптимизация.

Шаг 5. Начало обработки модификаторов и атрибутов показателей для выполнения вычислений. Для обработки используются таблицы, сформированные на шаге 4. В процессе обработки для каждого модификатора используются генератор, оптимизатор и синтаксический анализатор.

Шаг 6. Проверка наличия модификатора, при его отсутствии формирование части выходного SQL-выражения, включающего в себя данные текущего периода.

Шаг 7. В случае использования модификатора «B» формирование части выходного SQL-выражения, включающего в себя данные предыдущего временного среза за ближайший заполненный период.

Шаг 8. При использовании модификатора «E» формирование части выходного SQL-выражения, включающего в себя данные предыдущего периода в текущем временном срезе.

Шаг 9. В случае использования модификатора «Z» формирование части выходного SQL-выражения, включающего в себя данные текущего периода для всех сегментов используемой формы.

Шаг 10. В случае использования модификатора «EEP» формирование части выходного SQL-выражения, включающего в себя данные последнего периода предыдущего временного среза.

Шаг 11. В случае использования модификатора «PP» формирование части выходного SQL-выражения, включающего в себя данные периода, аналогичного текущему в предыдущем временном срезе.

Шаг 12. В случае использования модификатора «–1» формирование части выходного SQL-выражения, включающего в себя данные ближайшего заполненного периода текущего временного среза.

Шаг 13. В случае использования модификатора «–1D» формирование части выходного SQL-выражения, включающего в себя данные ближайшего заполненного периода.

Шаг 14. В случае использования модификатора «S_» формирование части выходного SQL-выражения, включающего в себя данные выбранного сегмента.

Шаг 15. В случае использования модификатора «смещение» формирование части выходного SQL-выражения, включающего в себя данные, смещенные на указанное количество периодов.

Шаг 17. В случае наличия модификатора, для которого невозможно определить его вид/тип, формирование исключения с сообщением об ошибке (синтаксический анализатор).

Шаг 18. Окончание обработки модификаторов и атрибутов показателей для выполнения вычислений.

Шаг 19. Сбор всех сгенерированных ранее частей SQL-выражений в одно целевое выражение на основании выражения, сформированного на шаге 4 (используются части выражений, сгенерированные на шагах 3, 6–17), – генератор и оптимизатор для целевого SQL-выражения.

Шаг 20. Завершение процесса генерации с передачей результата генерации (целевое SQL-выражение).

Теоретически реализация транслятора может быть выполнена на любом web-ориентированном языке программирования (php, python, ruby и др.) и любых фреймворках (Django и др.).

На основании изложенного можно сделать следующие выводы.

Описанный подход к построению многомерного аналитического пространства может быть использован для построения OLAP-кубов данных с помощью бесплатных свободно распространяемых программных средств построения web-приложений.

Суть подхода заключается в построении в бесплатной СУБД структуры БД по принципам ROLAP, разработке системы кодирования информационных объектов, языка обращения к многомерному пространству и транслятора, преобразующего выражения разработанного языка обращения к многомерному пространству в SQL-запросы бесплатной СУБД с помощью бесплатных программных средств web-разработки.

Описанный подход был разработан и реализован в рамках выполнения Федеральной целевой программы (Госконтракт № 14.527.12.0020 от 14

октября 2011 г.) по проекту «Программный комплекс и облачный сервис для автоматизации обобщения и анализа финансово-экономической информации о деятельности предприятий и обеспечения принятия управленческий решений» ЗАО «МБК Груп» совместно с ОАО «МХК «ЕвроХим» при финансовой поддержке Министерства образования и науки Российской Федерации.

Литература

1. Стариков А. Ядро OLAP-системы. Лаборатория BaseGroup. URL: http://www.basegroup.ru/ (дата обращения: 7.08.2013).

2. Барсегян А., Куприянов М., Степаненко В., Холод И. Технологии анализа данных. Data Mining, Text Mining, Visual Mining, OLAP. СПб: БХВ-Петербург, 2007. С. 19–58.

3. Niemi T., Nummenmaa J., Thanisch P. Proceeding DOLAP'01. Proc. 4th ACM intern. workshop on Data warehousing and OLAP, NY, 2001, pp. 9–15.

4. Арустамов А. Применение OLAP-технологий при извлечении данных. Лаборатория BaseGroup. URL: http://www.basegroup.ru/ (дата обращения: 7.08.2013).

5. Щавелев Л.В. Способы аналитической обработки данных для поддержки принятия решений // Системы управления базами данных. 1998. № 04–05.

6. Титоренко Г.А. Информационные технологии управления. М., 2003. С. 88–98.

7. Ильина О.П. Информационные технологии бухгалтерского учета. СПб: Питер, 2001. С. 57–60.

8. Назаров М.Г. Курс социально-экономической статистики. М., 2000. 771 с.

9. Карпов Ю.Г. Теория и технология программирования. Основы построения трансляторов. СПб: БХВ-Петербург, 2012.

10. Компаниец Р.И., Маньков Е.В., Филатов Н.Е. Системное программирование. Основы построения трансляторов. СПб: Корона-Принт, 2000.

11. Легалов А.И. Основы разработки трансляторов: учеб. пособие. Красноярск: Сибирский федер. ун-т, 2008.

12. Коробова И.Л., Дьяков И.А., Литовка Ю.В. Основы разработки трансляторов в САПР. ТГТУ, 2007.

13. Фридл Д. Регулярные выражения. СПб: Символ-Плюс, 2008.

14. Гойвертс Я., Левитан С. Регулярные выражения. Сборник рецептов. СПб: Символ-Плюс, 2010.

References

1. Starikov A. *The core of OLAP system*, BaseGroup, 2003, available at: http://www.basegroup.ru/ (accessed 26 August 2013) (in Russ.).

2. Barsegyan A., Kupriyanov M., Stepanenko V., Holod I. *Tekhnologii analiza dannykh. Data Mining, Text Mining, Visual Mining, OLAP* [Data analysis technologies. Data Mining, Text Mining, Visual Mining, OLAP]. St. Petersburg, BHV-Petersburg Publ., 2007, pp. 19–58 (in Russ.).

3. Niemi T., Nummenmaa J., Thanisch P. *Proc. of the 4th ACM int. workshop on Data warehousing and OLAP*. 2001, pp. 9–15.

4. Arustamov A. *Primenenye OLAP tekhnologiy pri izvlechenii dannykh* [OLAP technologies application when data retrieval]. BaseGroup, 2003, available at: http://www.basegroup.ru/ (accessed 26 August 2013) (in Russ.).

5. Shchavelev L.V. *Sistemy upravleniya bazami dannykh* [Databases management systems]. 1998, no. 04-05, p. 26 (in Russ.).

6. Titorenko G.A. *Informatsionnye tekhnologii upravleniya* [Information technologies of management]. 2nd ed., Moscow, 2003, pp. 88–98 (in Russ.).

7. Ilina O.P. *Informatsionnye tekhnologii bukhgalterskogo uchyota* [Information technologies of accounting]. St. Petersburg, Piter Publ., 2001, pp. 57–60 (in Russ.).

8. Nazarov M.G. *Kurs sotsialno-ekonomicheskoy statistiki* [Social-economic statistics course]. Moscow, 2000, 771 p. (in Russ.).

9. Karpov Yu.G. *Teoriya i tekhnologiya programmirovaniya. Osnovy postroeniya translyatorov* [Programming theory and technology. Fundamentals of translators]. BHV-Peterburg Publ., 2012 (in Russ.).

10. Kompaniets R.I., Mankov E.V., Filatov N.E. *Sistemnoe programmirovanie. Osnovy postroeniya translyatorov* [System programming. Fundamentals of translators]. St. Petersburg, Korona-Print Publ., 2000 (in Russ.).

11. Legalov A.I. *Osnovy razrabotki translyatorov* [Development basics of translators]. Textbook, Krasnoyarsk, Siberian Federal Univ. Publ., 2008 (in Russ.).

12. Korobova I.L., Dyakov I.A., Litovka Yu.V. *Osnovy razrabotki translyatorov v SAPR* [Development basics of translators in CAD system]. Tver State Tech. Univ. Publ., 2007 (in Russ.).

13. Friedl J. *Mastering Regular Expressions*. O'Reilly Media Publ., 2006.

14. Goyvaerts J., Levithan S. *Regular Expressions Cookbook*. O'Reilly Media Publ., 2009.

УДК 519.68

ВЕРИФИКАЦИЯ МУЛЬТИАГЕНТНЫХ СИСТЕМ С ПОМОЩЬЮ ЦЕПЕЙ МАРКОВА: ОЦЕНКА ВЕРОЯТНОСТИ НАХОЖДЕНИЯ АГЕНТАМИ ОПТИМАЛЬНОГО РЕШЕНИЯ

И.Д. Зайцев, аспирант
(Институт систем информатики им. А.П. Ершова СО РАН, просп. Лаврентьева, 6, г. Новосибирск, 630090, Россия, zaycev.ivan@gmail.com)

Предлагаются общее описание алгоритма работы определенного семейства мультиагентных систем и некоторые следующие из этого описания оценки свойств систем. В последнее время мультиагентный подход к построению сложных вычислительных систем обретает все большую популярность. Однако связанная с ним теория все еще недостаточно развита. Остаются неясными или недостаточно обоснованными вопросы о том, какого вида системы лучше применять в тех или иных случаях и в чем их преимущество перед другими методами. Отчасти это связано с тем, что в разных областях исследования под мультиагентной системой могут подразумеваться разные понятия. В данной работе выделяется и формализуется отдельное семейство мультиагентных систем, для полученной формализации проводится аналогия с марковскими цепями. Далее с помощью известных свойств марковских цепей исследуются свойства выделенного семейства систем. Для систем, построенных с помощью функции особого вида, доказы-